4. Project Design Phase

4.1 Introduction

The design phase transforms the requirements into a structured system design that defines the components, data flow, and functionality of LegalEase. The system follows a modular architecture consisting of a user interface, an application server, a database, and a Gemini AI integration module that communicate through well-defined APIs.

4.2 System Design

LegalEase follows a client-server architecture. The user interacts with the frontend, which sends requests to the FastAPI backend through REST APIs. The backend validates the form data, builds a prompt that contains the document type, jurisdiction, and user-supplied details, and calls the Google Gemini model to produce a structured draft. The draft is stored in the database and returned to the frontend for editing, review, and export. The Gemini API key is kept only on the server.

4.3 System Architecture

- **Presentation Layer:** login, registration, document selection, questionnaire, editor, explanation and review panel, export preview, and history pages.
- **Application Layer:** Python and FastAPI handling authentication, request validation, prompt construction, document assembly, and business logic.
- **AI Layer:** Google Gemini model for drafting clauses, explaining clauses, and reviewing documents.
- **Data Layer:** SQLite or PostgreSQL storing users, documents, clauses, and versions.

4.4 System Architecture Flow

User -> Frontend -> FastAPI Backend -> Prompt Builder -> Gemini Model -> Structured Draft (JSON) -> Database -> Editor and Review -> Export (PDF / Word) -> User

4.5 Module Design

Module	Design Description
Authentication	Registration, login with password hashing, JWT token issuing and verification, profile management, logout.
Document Selection and Input	Lists document types, shows the matching questionnaire, validates required fields, and sanitizes input.
AI Document Generator	Sends a structured prompt to Gemini and receives JSON containing the title, preamble, numbered clauses, and signature block; uses placeholders for missing data.
Clause Explanation	Sends a selected clause to Gemini and returns a plain-language explanation with each party's obligations.
Review and Risk Analysis	Analyses the draft for missing standard clauses, vague terms, and one-sided provisions and returns suggestions.
Editor and Version	Allows editing of the full document or a single clause, regeneration of a clause, and saving of versions.
Export	Assembles the final text into a formatted PDF using ReportLab or a Word file using python-docx.

Module	Design Description
Document History	Retrieves saved documents, allows opening, editing, duplicating, and deleting.

4.6 Database Design

Table	Main Fields
users	id, name, email, password_hash, default_jurisdiction, created_at
documents	id, user_id, doc_type, title, jurisdiction, input_json, status, created_at
clauses	id, document_id, clause_no, heading, content
versions	id, document_id, version_no, full_text, created_at
reviews	id, document_id, issues_json, created_at
generation_log	id, user_id, document_id, type (draft/explain/review), status, created_at

4.7 API Design

Method	Endpoint	Purpose
POST	/auth/register	Create a new user account
POST	/auth/login	Authenticate and return JWT
GET	/templates	List supported document types and their questions
POST	/documents	Create a document from user inputs and generate the draft
GET	/documents	List the user's saved documents
GET / PUT / DELETE	/documents/{id}	View, edit, or delete a document
POST	/documents/{id}/clauses/{n}/regenerate	Regenerate a single clause
POST	/documents/{id}/clauses/{n}/explain	Explain a clause in plain language
POST	/documents/{id}/review	Review the document for gaps and risks
GET	/documents/{id}/export	Export the document as PDF or Word

4.8 Security Design

- Passwords are stored as hashes, never as plain text.
- JWT tokens protect all private endpoints, and each query is filtered by user_id.
- The Gemini API key is stored in environment variables on the server.
- User input is validated and sanitized before being sent to the AI model.
- Gemini safety settings and output checks are used to block unsafe or unlawful requests.
- Documents are served only to their owner and are not shared with other users.